

Final Report: Camera fault detection for Autonomous Delivery Vehicles

Liu Yuchen, Wu Zekai, Shi Xuanwei

2026 年 1 月 18 日

1 Task Description and Main Difficulty

This project addresses camera fault detection for Autonomous Delivery Vehicles (ADV) by formulating it as a multi-class health-state classification problem. For each onboard camera, the input corresponds to one measurement cycle consisting of 60 time steps of three telemetry time series: Current (mA), GPU usage (%), and CPU usage (%), together with three cycle-level scalar indicators: EyeQuality, DecodeErrCount, and miniFps.

The output is a discrete health label with four classes: (1) Normal operation, (2) Warning, (3) Degraded performance, (4) Shutdown or unreliable output.

The goal is to design accurate classification algorithms that can reliably assess camera health from runtime data and to analyze how feature engineering choices affect performance.

Key challenges include the time dependency within each cycle, heterogeneous feature types, and real-world telemetry issues such as noise and outliers (e.g., spikes in current or utilization) and potential irregular readings. Additionally, the four health states may be class-imbalanced in practice (fault states rarer than normal), which can bias standard training toward the majority class if not handled carefully.

2 Data Preprocessing

2.1 Preprocessing of cross-section data

For the cross-sectional data, considering that their mean and variance have significant differences, in order to make the calculation more accurate and facilitate better training of the model, we standardized the three sets of cross-sectional data.

2.2 Preprocessing of time series data

For time series data, due to its characteristics of both time-varying and cumulative nature, two types of processing are carried out on the time series data: calculation of basic statistics and calculation of first-order differences.

In terms of basic statistics, we calculated the arithmetic mean, standard deviation, maximum value, minimum value, range, slope (linear regression between time and value), kurtosis and skewness for Current, cpuUsage and gpuUsage at each of the 60 observation points for each sample (Table 1, Table 2), and used these as new candidate variables for model training.

For the processing of first-order differences, for each sample, we successively calculate the differences between two observation points of the three time series data, obtaining 59 differential data to reflect the fluctuation of the

feature	mean	std	max	min
current	Current_mean	Current_std	Current_max	Current_min
GPUusage	gpuUsage_mean	gpuUsage_std	gpuUsage_max	gpuUsage_min
CPUusage	cpuUsage_mean	cpuUsage_std	cpuUsage_max	cpuUsage_min

Table 1: Basic Statistics (1)

feature	slope	skew	kurt	special
current	Current_slope	Current_skewness	Current_kurtosis	current_high_count
GPUusage	gpuUsage_slope	gpuUsage_skewness	gpuUsage_kurtosis	gpu_high_count
CPUusage	cpuUsage_slope	cpuUsage_skewness	cpuUsage_kurtosis	cpu_high_count

Table 2: Basic Statistics (2)

equipment’s operation. If there is a significant change between two time points, it may be reflected in the basic statistics(same as Table 1) of first-order difference values of the three time series data. In reality, those equipment whose working parameters change relatively smoothly tend to have a better fault state.

In addition to the main attempt, we also constructed other indicators, such as the number of observations where cpuUsage is greater than 50% (cpu_high_count), the number of observations where gpuUsage is greater than 70% (gpu_high_count), and the number of observations where current is greater than 95mA (current_high_count).

3 Model Building & Methodology

3.1 Model Selection Rationale

The dataset is characterized by mixed feature types, feature collinearity, class imbalance, high dimensionality, and difficulty in distinguishing adjacent health levels. Guided by requirements for low complexity, interpretability, and strong generalization, Random Forest (RF) was selected as the core model, with LightGBM as a supplementary model and an ensemble strategy to enhance performance.

Random Forest[1] was chosen for three key advantages: First, it naturally handles mixed feature types without requiring strict normalization, preserving information from both time-series and scalar features while avoiding information loss from scaling. Second, as an ensemble model integrating multiple decision trees, it reduces overfitting through random feature selection and tree diversification, which is crucial for the high-dimensional feature space (over 50 features post-preprocessing). Third, it provides inherent feature importance evaluation, enabling efficient feature screening to address collinearity.

LightGBM[2] was introduced to mitigate class imbalance, leveraging its gradient-based one-side sampling and built-in scale_pos_weight parameter. Four additional models (Gradient Boosting, AdaBoost, Decision Tree, KNN) were implemented for benchmarking, providing parallel comparisons for the core models.

3.2 Model Architecture

The RF model takes a 12-dimensional optimized feature matrix (selected via importance ranking) as input and outputs discrete health labels (1-4). Key components include the number of trees, maximum depth, and minimum samples for splitting, optimized through grid search. The LightGBM model uses the same feature input with `num_class=4`, `learning_rate=0.05`, and `class_weight='balanced'` to handle imbalance, outputting class probabilities. A weighted ensemble combined RF and LightGBM predictions, with optimal weights determined via 5-fold stratified cross-validation to maximize the weighted F1-score.

4 Model Training

4.1 Training Pipeline

Data splitting adopted Time Series Cross-Validation combined with 8-fold cross-validation to preserve temporal order and prevent data leakage, ensuring the model generalizes to unseen sequential data. Feature selection utilized RF's importance ranking: the full preprocessed feature matrix (including statistical features like mean/std of time-series data and first-order differentiation) was input to RF, features were ranked by importance, and the top 12 were selected to reduce collinearity and overfitting. Class imbalance was addressed via SMOTE (Synthetic Minority Oversampling Technique)[3], which generates synthetic minority samples while preserving feature distribution, complemented by balanced class weights in both models.

4.2 Hyperparameter Tuning

Grid Search was used for RF tuning, with the search space as follows:

Parameter	Range
Max_depth	[None, 10, 20, 30]
Min_sample_split	[2, 5, 10]
n_estimators	[100, 200, 300]

Table 3: Search space of RF hyperparameters

Best Parameters from Grid Search are `{'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 200}`. The ensemble weight for RF was searched in `[0.0, 1.0]` at 0.1 intervals via Random Search, with cross-validation to identify the optimal value.

4.3 Training Details

Training used a fixed random seed (42) for reproducibility. Regularization was implemented via tree depth limitation and minimum sample criteria. The computing environment supported efficient execution, meeting real-time requirements. The 8-fold CV mean score for RF was 0.95, confirming robustness. The final training pipeline included data preprocessing, feature selection, model training, hyperparameter tuning, and ensemble fusion, ensuring a systematic and reproducible process.

5 Results

The accuracy of different machine-learning models are as follows:

Models	Random Forest	Gradient Boosting	Decision Tree	Adaboost	KNN
Accuracy	0.88	0.91	0.83	0.71	0.84

Table 4: Accuracy of different models

After solving the problem of data imbalance by using the SMOTE method and integrating the lightGBM model, the comparison of the obtained results with those without processing is shown in the following table.

	4_recall	3_precession	3_f1-score	4_f1-score	Accuracy
Before	0.75	0.69	0.75	0.85	0.87
After	0.81	0.71	0.79	0.87	0.88

Table 5: Comparison of whether solving data imbalance

6 Evaluation

Given the class imbalance in the dataset, we designed a comprehensive evaluation framework that uses overall accuracy, F1-score for Class 3, and F1-score for Class 4 as the core metrics. We trained and validated the candidate models (Random Forest, AdaBoost, Decision Tree, KNN, and Gradient Boosting) using these metrics, and their performance on the validation set is shown in [Figure 1](#).

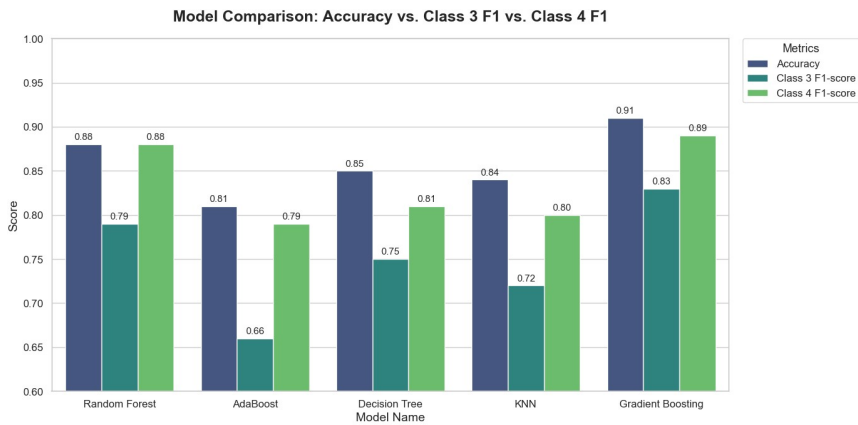


Figure 1: Performance for models

Among these models, Gradient Boosting achieved the best overall performance, with Random Forest as the second-best performer. The top 15 important features and the confusion matrices (on the validation set) for these two models are presented in [Figure 2](#), [Figure 3](#):

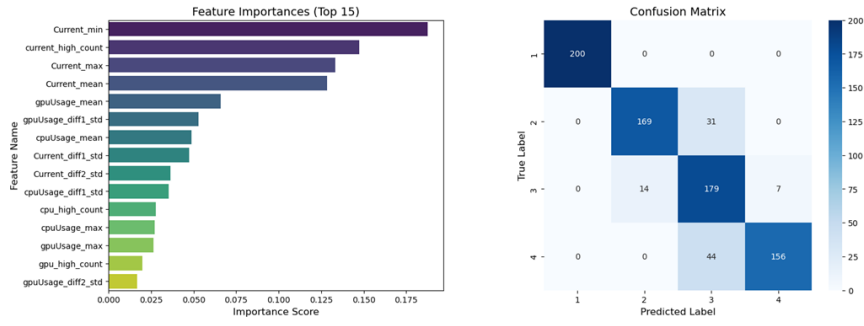


Figure 2: Random Forest

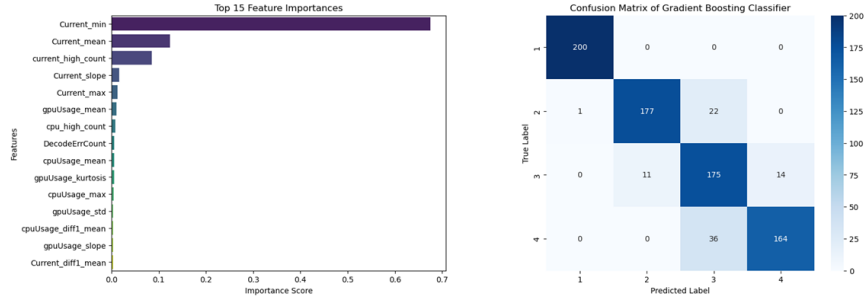


Figure 3: Gradient Boosting

7 Analysis & Discussion

7.1 Performance Comparison

Based on the comprehensive evaluation, the Gradient Boosting Classifier emerged as the optimal model, achieving the highest scores across all key metrics. The Random Forest Classifier ranked second, delivering competitive results with only a slight margin of difference.

7.2 Efficiency Analysis and Final Decision

Both models perform exceptionally well because they utilize ensemble learning to capture complex, non-linear relationships within the high-dimensional data. However, their training mechanisms differ fundamentally:

- **Gradient Boosting (Sequential):** Uses a boosting strategy where trees are built sequentially to correct the residuals (errors) of previous trees. While this minimizes bias, it prevents parallelization, resulting in low training efficiency. The training time is often several times longer than that of Random Forest.
- **Random Forest (Parallel):** Uses a bagging strategy where trees are built independently and can be trained in parallel.

Although Gradient Boosting provided a marginal performance gain, it came at the cost of significant computational overhead. To balance predictive accuracy with operational efficiency, Random Forest was selected as the final model for deployment.

7.3 Divergence in Feature Selection

A distinct difference was observed in how the two models rank feature importance:

- **Gradient Boosting (Greedy Approach):** The model exhibits a "greedy" selection process. Once a strong feature is selected to minimize errors, correlated features are often ignored (masked). This results in a distribution where a few top features have very high importance, while the rest drop off sharply.
- **RandomForest (Robust Approach):** Due to its random subspace method, importance is "shared" among correlated features. This results in a smoother distribution of importance scores, reflecting a more holistic view of feature robustness rather than just error minimization.

8 LSTM

Besides the traditional machine learning models mentioned above, we also tried to use LSTM to solve this classification problem.

8.1 Data Preprocessing[4]

For the LSTM approach, we built a compact set of 17 step-wise features to preserve temporal information while keeping the input dimension small to reduce overfitting. At each of the 60 time steps, the feature vector includes: (i) the raw signals [Current, gpuUsage, cpuUsage], (ii) first-order differences of the three signals to represent short-term changes, (iii) deviations from the cycle mean to capture within-cycle abnormality, (iv) pairwise interaction terms (Current x gpuUsage, Current x cpuUsage, gpuUsage x cpuUsage) to model coupled load patterns, (v) a sinusoidal time encoding (sin, cos) to provide position information and (vi) the three scalar indicators [EyeQuality, DecodeErrCount, miniFps] repeated across time steps. Additionally, we applied robust clipping based on training-set quantiles (0.5% to 99.5%) and standardized all features using training statistics.

8.2 Model Building & Methodology

We adopt an attention-based bidirectional LSTM (AttnBiLSTM) as the main model to leverage both past and future context within each 60-step cycle. Given an input sequence X in $R^{60 \times 17}$, a 2-layer BiLSTM encoder produces hidden states h_t in R^{2H} . A lightweight attention module then assigns a weight to each time step and computes a context vector as the weighted sum of hidden states, which is finally fed into a feed-forward classification head to output four-class logits. Weighted cross-entropy loss is used for multi-class training.

8.3 Model Training

The model is optimized with AdamW (initial learning rate 1e-3, weight decay 1e-2) and trained for up to 50 epochs. To stabilize training, we apply gradient clipping (max norm 1.0). A ReduceLROnPlateau scheduler (mode=max on validation accuracy, factor=0.5, patience=3) is used to automatically reduce the learning rate when improvement plateaus. Because fault-related classes can be under-represented, we use class-weighted cross-entropy where weights are inversely proportional to class frequencies in the training subset. The final model is selected by the best validation accuracy, and the corresponding checkpoint is evaluated on the test set.

8.4 Results & Evaluation

We evaluate the final LSTM model using accuracy and per-class precision/recall/F1 on the held-out test set. The best validation accuracy reaches 0.9925, and the final test accuracy is 0.9063. Class-wise results indicate that

the model performs extremely well on Class 1 , while most errors occur between neighboring fault severities (Class 2 to Class 3 and Class 3 to Class 4), which is consistent with the gradual transition of camera health states.

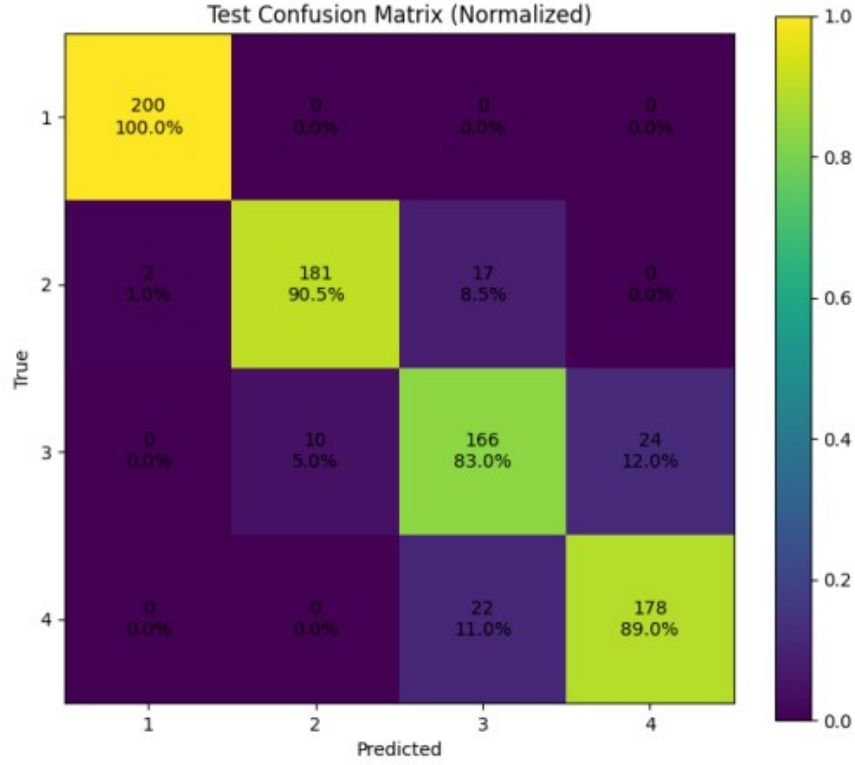


Figure 4: LSTM Confusion Matrix

As shown in Figure 4, Class 1 is classified perfectly. The main confusion arises between Class 2 and Class 3, and between Class 3 and Class 4, suggesting that borderline degradation stages share similar telemetry signatures.

9 References

- [1] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] G. Ke et al., “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.
- [4] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, “Long short term memory networks for anomaly detection in time series,” in *Proceedings of ESANN*, 2015.